

Item-restscore power with the conditional DGP

Follow-up to `restscore_infit_power.qmd`, reduced design

Magnus Johansson

2026-10-03

Table of contents

Purpose	1
Design	2
One dataset	3
Run	4
Datasets that failed in the earlier study	5
Paired data	6
Results	6
Detection of the misfitting items	6
Paired comparisons	9
False flags on the fitting items, by direction	10
Clean decisions	11
Pooled detection	12
Iterations kept	12
Rescued datasets	13
Interpretation	13
Follow-up: split one-sided step-downs	14
Checks	18
Detection of the misfitting items	18
Paired comparisons	20
False flags on the fitting items, by direction	21
Clean decisions	22
Extension to 300 datasets per cell	23
Interpretation of the split follow-up	28

Purpose

`restscore_infit_power.qmd` compared item-restscore with `infit` under planted misfit, with both cutoff functions at their defaults at the time (`dgp = "resample"`). After that study and the validation study (`restscore_cutoff_validation.qmd`), `RMitemRestscoreCutoff()` changed in two ways:

1. The default DGP is now `"conditional"`, which held the nominal family-wise rate under the null where `"resample"` was about 1.4 points liberal.
2. The bootstrap p-values in `RMitemRestscore()` are equal-tailed. Folding $|t|$ at the mean gave the long underfit tail more than $\alpha/2$.

This study measures what the shipped configuration does under misfit, and separates the two changes. Each restscore cutoff is simulated once with the conditional DGP, and the p-values are computed from the same simulation both ways, equal-tailed (as shipped) and $|t|$ (as before). Comparing

- conditional $|t|$ with the earlier resample $|t|$ isolates the DGP, and
- conditional equal-tailed with conditional $|t|$ isolates the p-value.

`Infit` is unchanged, so its results are reused from the earlier study, as is the asymptotic test.

Design

Identical to `restscore_infit_power.qmd` (7 items, 5 categories, underfit by a second dimension at $p = .85$, overfit by slope 1.6, 1 or 2 misfitting items at 0 or -2 logits, 100 datasets per cell, $B = 400$), with two exceptions:

- **Sample size** 150 and 400 only, 16 of the 32 cells.
- **Methods** run here: `RMitemRestscoreCutoff(dgp = "conditional")`, then WY and BH, each with the equal-tailed and the $|t|$ p-value.

The datasets are regenerated from the same seeds, so every comparison with the earlier study is paired. In the earlier study 57 datasets failed in both cutoff functions, because a category was used only by respondents with a zero or perfect total score (all of them in the $n = 150$ and 250 cells, and one at $n = 400$). The conditional DGP reproduces those respondents' patterns, so the restscore cutoff is expected to run on them. They are counted, but left out of the comparisons with `infit`, which has no result for them.

Cost. About 30 to 45 s per dataset for the conditional restscore cutoff alone, from the timings in the validation study. 1,600 datasets is then roughly 1.5 to 2 hours on 10 cores. Results are saved per cell, so an interrupted render resumes. Set `R_DATASETS` lower for a quick run.

```
suppressMessages({
  devtools::load_all("...", quiet = TRUE)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})
options(rgl.useNULL = TRUE)

# The study depends on the conditional default; stop rather than silently
# measure the old behaviour. (It also checked for the equal-tailed p-value,
# which has since been withdrawn; the cached cond_eq_* results came from it.)
stopifnot(
  "conditional is not the RMitemRestscoreCutoff() default" =
    identical(eval(formals(RMitemRestscoreCutoff)$dgp)[1], "conditional")
)

R_DATASETS <- 100
B          <- 400
RHO        <- 0.85
SLOPE      <- 1.6
NCORES     <- 10
NS         <- c(150, 400)
OLD_DIR    <- "restscore_infit_power"
RES_DIR    <- "restscore_power_conditional"
dir.create(RES_DIR, showWarnings = FALSE)

source("restscore_power_generator.R")

# The full grid of the earlier study, in its order: the seed of a dataset is
# 1e6 * (its row in this grid) + rep, so the grid must not be reordered.
cells <- expand_grid(
  n = c(150, 250, 400, 600),
  target = c(0, -2),
  n_misfit = c(1, 2),
  direction = c("underfit", "overfit"),
```

```

    stringsAsFactors = FALSE
  )
  cells$cell <- with(cells, sprintf("%s_%ditem_t%s_n%d", direction, n_misfit,
                                   ifelse(target == 0, "0", "m2"), n))
  cells$row <- seq_len(nrow(cells))
  run_cells <- cells[cells$n %in% NS, ]
  nrow(run_cells)

```

```
[1] 16
```

One dataset

```

one_dataset <- function(rep, cell_row) {
  setup <- item_setup(cell_row$n_misfit, cell_row$target)
  seed <- 1e6 * cell_row$row + rep
  d <- sim_power_data(
    n = cell_row$n, n_misfit = cell_row$n_misfit, target = cell_row$target,
    direction = cell_row$direction, rho = RHO, slope = SLOPE, seed = seed
  )

  tryCatch({
    cr <- suppressWarnings(suppressMessages(RMitemRestscoreCutoff(
      d, iterations = B, parallel = FALSE, seed = seed,
      cutoff_method = "quantile", dgp = "conditional"
    )))
    q <- function(expr) suppressWarnings(suppressMessages(expr))
    flags <- function(tab) tab$Flagged[match(names(d), tab$Item)]
    eq_wy <- q(RMitemRestscore(d, cutoff = cr, output = "dataframe"))
    eq_bh <- q(RMitemRestscore(d, cutoff = cr, correction = "fdr_bh",
                              output = "dataframe"))

    # The |t| p-values from the same simulation, flagged the way
    # RMitemRestscore() did before the equal-tailed change.
    sim_mat <- tapply(cr$results$Difference,
                      list(cr$results$iteration, cr$results$Item),
                      function(x) x[1L])
    obs <- stats::setNames(eq_wy$Difference, eq_wy$Item)
    null_mean <- colMeans(sim_mat, na.rm = TRUE)[names(obs)]
    abs_flags <- function(correction) {
      pv <- .bootstrap_pvalues(obs, sim_mat, correction = correction,
                              tail = "two.sided")
      padj <- pv$padj[match(names(obs), pv$name)]
      f <- ifelse(!is.na(padj) & padj < .05,
                  ifelse(obs > null_mean, "overfit", "underfit"), "")
      f[match(names(d), names(obs))]
    }

    data.frame(
      rep = rep,
      item = names(d),
      misfit = seq_len(7) %in% setup$misfit,
      cond_eq_wy = flags(eq_wy),
      cond_eq_bh = flags(eq_bh),

```

```

    cond_abs_wy = abs_flags("fwer"),
    cond_abs_bh = abs_flags("fdr_bh"),
    iterations = cr$actual_iterations,
    stringsAsFactors = FALSE
  )
}, error = function(e) {
  data.frame(rep = rep, error = conditionMessage(e))
})
}

```

Errors are kept as rows rather than returned as NULL, so a failure is visible in the results instead of looking like a missing dataset.

Run

```

for (i in seq_len(nrow(run_cells))) {
  f <- file.path(RES_DIR, paste0(run_cells$cell[i], ".rds"))
  if (file.exists(f)) next
  t0 <- Sys.time()
  res <- mclapply(seq_len(R_DATASETS),
    function(r) one_dataset(r, run_cells[i, ]),
    mc.cores = NCORES, mc.preschedule = FALSE)
  saveRDS(list(res = res,
    elapsed_min = as.numeric(Sys.time() - t0, units = "mins")),
    f)
}

read_cell <- function(dir, cell) {
  x <- readRDS(file.path(dir, paste0(cell, ".rds")))
  x$res
}

# New results: successful datasets and errors kept apart.
new_raw <- lapply(run_cells$cell, function(cl) {
  res <- read_cell(RES_DIR, cl)
  is_ok <- vapply(res, function(x) is.data.frame(x) && "item" %in% names(x),
    logical(1))
  list(ok = do.call(rbind, res[is_ok]),
    err = do.call(rbind, res[!is_ok]))
})
new <- do.call(rbind, Map(function(x, cl) cbind(cell = cl, x$ok),
  new_raw, run_cells$cell))
errors <- do.call(rbind, Map(function(x, cl) {
  if (is.null(x$err)) NULL else cbind(cell = cl, x$err)
}, new_raw, run_cells$cell))

# Earlier results (infit, resample restscore with |t|, asymptotic).
old <- do.call(rbind, lapply(run_cells$cell, function(cl) {
  res <- read_cell(OLD_DIR, cl)
  ok <- Filter(Negate(is.null), res)
  cbind(cell = cl, do.call(rbind, ok))
})) |>
  rename(rest_resample_abs_wy = rest_boot_wy,
    rest_resample_abs_bh = rest_boot_bh)

```

```
run_cells |>
  select(cell) |>
  left_join(count(distinct(new, cell, rep), cell, name = "new_ok"),
            by = "cell") |>
  left_join(count(distinct(old, cell, rep), cell, name = "old_ok"),
            by = "cell") |>
  mutate(new_errors = vapply(cell, function(cl) {
    if (is.null(errors)) 0L else sum(errors$cell == cl)
  }, integer(1)),
         minutes = vapply(cell, function(cl) {
    round(readRDS(file.path(RES_DIR, paste0(cl, ".rds")))$elapsed_min, 1)
  }, numeric(1)))
```

cell	new_ok	old_ok	new_errors	minutes
underfit_1item_t0_n150	100	100	0	5.0
underfit_1item_t0_n400	100	100	0	6.2
underfit_1item_tm2_n150	100	95	0	4.8
underfit_1item_tm2_n400	100	100	0	6.2
underfit_2item_t0_n150	100	100	0	5.0
underfit_2item_t0_n400	100	100	0	6.2
underfit_2item_tm2_n150	100	96	0	4.7
underfit_2item_tm2_n400	100	100	0	6.2
overfit_1item_t0_n150	100	99	0	5.0
overfit_1item_t0_n400	100	100	0	6.2
overfit_1item_tm2_n150	100	90	0	4.8
overfit_1item_tm2_n400	100	99	0	6.2
overfit_2item_t0_n150	100	99	0	5.1
overfit_2item_t0_n400	100	100	0	6.2
overfit_2item_tm2_n150	100	86	0	4.7
overfit_2item_tm2_n400	100	100	0	6.2

```
if (!is.null(errors)) count(errors, error)
```

Datasets that failed in the earlier study

```
rescued <- new |>
  distinct(cell, rep) |>
  anti_join(distinct(old, cell, rep), by = c("cell", "rep"))
nrow(rescued)
```

```
[1] 36
```

```
count(rescued, cell)
```

cell	n
overfit_1item_t0_n150	1
overfit_1item_tm2_n150	10
overfit_1item_tm2_n400	1
overfit_2item_t0_n150	1
overfit_2item_tm2_n150	14
underfit_1item_tm2_n150	5
underfit_2item_tm2_n150	4

Paired data

Comparisons below use only datasets with results in both studies.

```
methods <- c("infit_wy", "rest_resample_abs_wy", "cond_abs_wy", "cond_eq_wy",
             "infit_bh", "rest_resample_abs_bh", "cond_abs_bh", "cond_eq_bh",
             "rest_asy_bh")

both <- inner_join(
  old |> select(cell, rep, item, misfit, infit_wy, infit_bh,
               rest_resample_abs_wy, rest_resample_abs_bh, rest_asy_bh),
  new |> select(cell, rep, item, cond_eq_wy, cond_eq_bh, cond_abs_wy,
               cond_abs_bh),
  by = c("cell", "rep", "item")
) |>
  left_join(run_cells |> select(cell, n, target, n_misfit, direction),
            by = "cell")

long <- both |>
  pivot_longer(all_of(methods), names_to = "method", values_to = "flag") |>
  mutate(flag = ifelse(is.na(flag), "", flag),
         method = factor(method, levels = methods),
         targeting = ifelse(target == 0, "misfit at 0", "misfit at -2"),
         misfit_items = paste(n_misfit, ifelse(n_misfit == 1, "item", "items")))
```

Results

Detection of the misfitting items

Proportion of misfitting items flagged in the correct direction, WY methods first, then BH and the asymptotic test.

```
det <- long |>
  filter(misfit) |>
  group_by(direction, misfit_items, targeting, n, method) |>
  summarise(correct = round(100 * mean(flag == direction)),
            .groups = "drop")

det |>
  pivot_wider(names_from = method, values_from = correct) |>
  arrange(direction, misfit_items, desc(targeting), n)
```

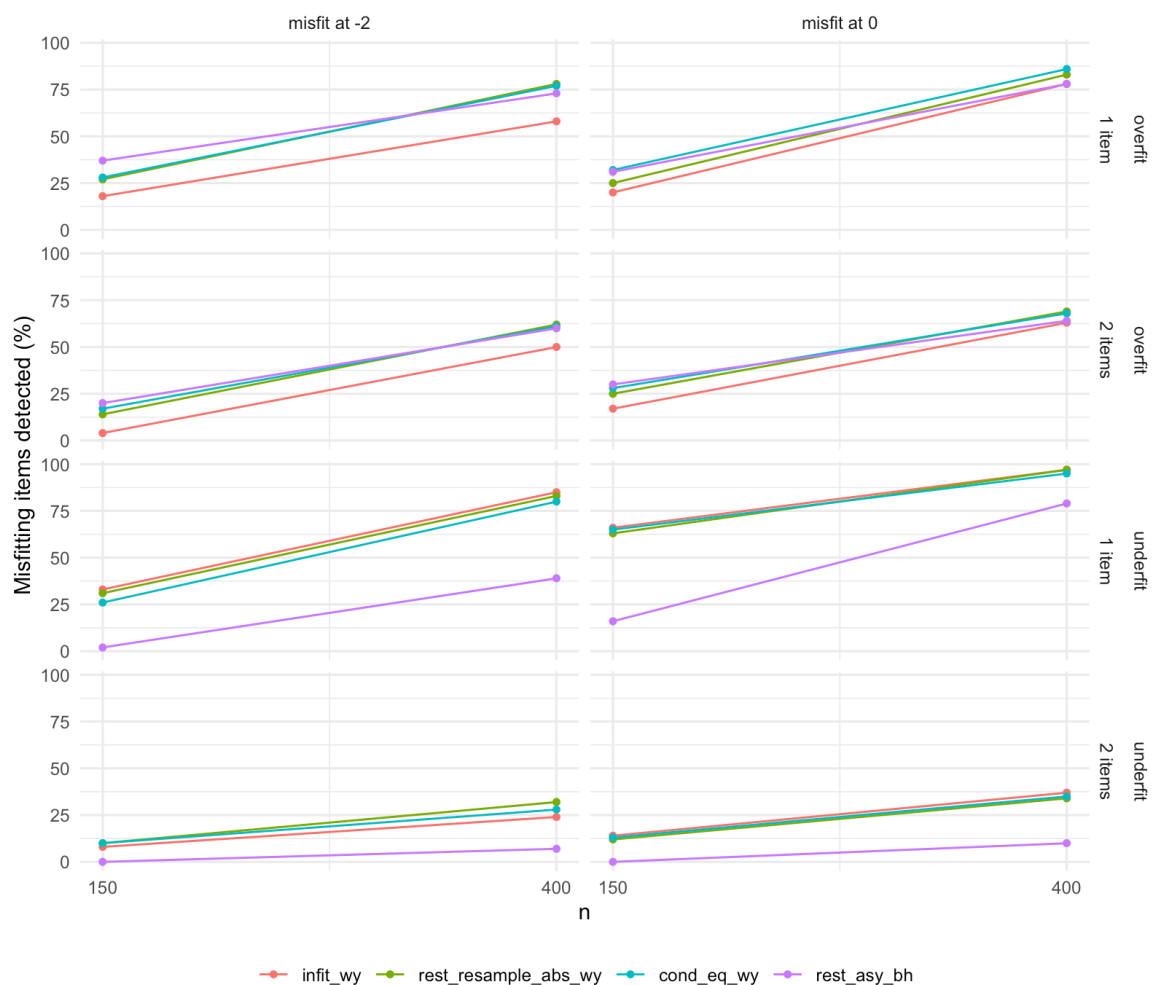
direction	misfit_items getting	trans misfit_items	sample_absonly	absonly	absonly_eq_vis	fitres	sample_absonly	absonly	absonly_eq_vis	res	asy_bh	
over-fit	1 item	mis-fit at 0	150	20	25	29	32	16	19	23	25	31
over-fit	1 item	mis-fit at 0	400	78	83	84	86	67	75	80	82	78
over-fit	1 item	mis-fit at -2	150	18	27	19	28	16	18	12	13	37
over-fit	1 item	mis-fit at -2	400	58	78	76	77	53	71	68	69	73
over-fit	2 items	mis-fit at 0	150	17	25	24	28	19	24	24	24	30
over-fit	2 items	mis-fit at 0	400	63	69	68	68	69	75	72	68	64
over-fit	2 items	mis-fit at -2	150	4	14	12	17	7	12	9	12	20
over-fit	2 items	mis-fit at -2	400	50	62	56	61	50	64	63	58	60
under-fit	1 item	mis-fit at 0	150	66	63	61	65	56	55	59	52	16
under-fit	1 item	mis-fit at 0	400	97	97	97	95	97	96	93	90	79
under-fit	1 item	mis-fit at -2	150	33	31	26	26	31	28	24	21	2
under-fit	1 item	mis-fit at -2	400	85	83	83	80	80	75	78	68	39
under-fit	2 items	mis-fit at 0	150	14	12	12	13	11	10	12	8	0
under-fit	2 items	mis-fit at 0	400	37	34	34	35	34	34	34	33	10

direction	misfit_items	targeting	infit_wy	rest_resample_abs_wy	cond_eq_wy	rest_asy_bh	infit_wy	rest_resample_abs_wy	cond_eq_wy	rest_asy_bh	
under-2 fit	mis-150 items	fit at -2	8	10	10	10	8	10	7	6	0
under-2 fit	mis-400 items	fit at -2	24	32	29	28	24	29	30	26	7

```

det |>
  filter(method %in% c("infit_wy", "rest_resample_abs_wy", "cond_eq_wy",
    "rest_asy_bh")) |>
  ggplot(aes(n, correct, colour = method)) +
  geom_line() +
  geom_point(size = 1.2) +
  facet_grid(direction + misfit_items ~ targeting) +
  scale_x_continuous(breaks = NS) +
  labs(x = "n", y = "Misfitting items detected (%)", colour = NULL) +
  theme_minimal() +
  theme(legend.position = "bottom")

```



Paired comparisons

Misfitting items detected by one method and not the other, pooled over sample size and the number of misfitting items. `difference_pts` is the first method minus the second.

```
paired <- function(a, b) {
  both |>
  filter(misfit) |>
  mutate(hit_a = .data[[a]] == direction, hit_b = .data[[b]] == direction,
         hit_a = !is.na(hit_a) & hit_a, hit_b = !is.na(hit_b) & hit_b) |>
  group_by(direction, targeting = ifelse(target == 0, "at 0", "at -2")) |>
  summarise(items = n(),
            a_only = sum(hit_a & !hit_b),
            b_only = sum(!hit_a & hit_b),
            difference_pts = round(100 * (mean(hit_a) - mean(hit_b)), 1),
            .groups = "drop") |>
  mutate(comparison = paste(a, "vs", b), .before = 1)
}
bind_rows(
  paired("cond_abs_wy", "rest_resample_abs_wy"), # the DGP
  paired("cond_abs_wy", "infit_wy"),             # shipped restscore vs infit
  paired("cond_abs_wy", "rest_asy_bh"),          # shipped restscore vs iarm
  paired("cond_eq_wy", "cond_abs_wy")           # withdrawn equal-tailed version
)
```

comparison	direction	targeting	items	a_only	b_only	difference_pts
cond_abs_wy vs rest_resample_abs_wy	overfit	at -2	561	6	31	-4.5
cond_abs_wy vs rest_resample_abs_wy	overfit	at 0	597	13	11	0.3
cond_abs_wy vs rest_resample_abs_wy	underfit	at -2	587	5	13	-1.4
cond_abs_wy vs rest_resample_abs_wy	underfit	at 0	600	7	9	-0.3
cond_abs_wy vs infit_wy	overfit	at -2	561	71	27	7.8
cond_abs_wy vs infit_wy	overfit	at 0	597	49	8	6.9
cond_abs_wy vs infit_wy	underfit	at -2	587	41	33	1.4
cond_abs_wy vs infit_wy	underfit	at 0	600	11	24	-2.2
cond_abs_wy vs rest_asy_bh	overfit	at -2	561	9	46	-6.6
cond_abs_wy vs rest_asy_bh	overfit	at 0	597	26	23	0.5
cond_abs_wy vs rest_asy_bh	underfit	at -2	587	131	0	22.3
cond_abs_wy vs rest_asy_bh	underfit	at 0	600	136	0	22.7
cond_eq_wy vs cond_abs_wy	overfit	at -2	561	39	10	5.2
cond_eq_wy vs cond_abs_wy	overfit	at 0	597	23	13	1.7
cond_eq_wy vs cond_abs_wy	underfit	at -2	587	14	18	-0.7
cond_eq_wy vs cond_abs_wy	underfit	at 0	600	14	10	0.7

False flags on the fitting items, by direction

```
long |>
  filter(!misfit) |>
  group_by(direction, misfit_items, method) |>
  summarise(false_overfit = round(100 * mean(flag == "overfit"), 1),
            false_underfit = round(100 * mean(flag == "underfit"), 1),
            .groups = "drop") |>
  pivot_wider(names_from = misfit_items,
              values_from = c(false_overfit, false_underfit))
```

di- rec- tion	method	false_overfit_1 item	false_overfit_2 items	false_underfit_1 item	false_underfit_2 items
over- fit	infit_wy	0.0	0.0	1.5	6.5
over- fit	rest_resample_abs_wy	0.1	0.0	1.5	6.3
over- fit	cond_abs_wy	0.1	0.0	1.4	5.9
over- fit	cond_eq_wy	0.2	0.1	1.4	5.5
over- fit	infit_bh	0.2	0.1	2.0	8.9
over- fit	rest_resample_abs_bh	0.2	0.0	1.8	8.9
over- fit	cond_abs_bh	0.2	0.1	1.8	7.7
over- fit	cond_eq_bh	0.1	0.1	1.3	6.6
over- fit	rest_asy_bh	0.2	0.1	0.0	0.8
un- derfit	infit_wy	1.1	1.2	0.0	0.1
un- derfit	rest_resample_abs_wy	2.3	2.7	0.0	0.1
un- derfit	cond_abs_wy	1.9	2.4	0.0	0.1
un- derfit	cond_eq_wy	2.3	3.3	0.0	0.3
un- derfit	infit_bh	1.5	2.0	0.1	0.1
un- derfit	rest_resample_abs_bh	2.8	3.7	0.1	0.1
un- derfit	cond_abs_bh	2.3	3.4	0.1	0.1

di- rec- tion	method	false_overfit_1 item	false_overfit_2 items	false_underfit_1 item	false_underfit_2 items
un- derfit	cond_eq_bh	2.1	3.1	0.0	0.2
un- derfit	rest_asy_bh	2.2	2.2	0.0	0.0

Clean decisions

Every misfitting item flagged in the correct direction and no fitting item flagged.

```
long |>
  group_by(direction, misfit_items, targeting, n, method, rep) |>
  summarise(clean = all(flag[misfit] == direction[1]) &
    all(flag[!misfit] == ""),
    .groups = "drop") |>
  group_by(direction, misfit_items, targeting, n, method) |>
  summarise(clean = round(100 * mean(clean)), .groups = "drop") |>
  pivot_wider(names_from = method, values_from = clean) |>
  arrange(direction, misfit_items, desc(targeting), n)
```

direction	misfit_items	targeting	n	method	rep	cond_eq_bh	rest_asy_bh					
overfit	1 item	misfit at 0	150	18	23	25	27	12	15	17	20	31
overfit	1 item	misfit at 0	400	66	72	73	76	52	62	64	71	78
overfit	1 item	misfit at -2	150	18	22	16	24	12	13	9	10	36
overfit	1 item	misfit at -2	400	52	71	70	73	42	61	58	65	73
overfit	2 items	misfit at 0	150	1	3	2	5	2	2	4	5	11
overfit	2 items	misfit at 0	400	11	18	21	17	10	14	17	12	36
overfit	2 items	misfit at -2	150	0	2	1	1	2	2	0	1	6
overfit	2 items	misfit at -2	400	12	21	16	21	9	18	15	15	38

direction	misfit_items_getting	trans_res	resample_abscond	cond_abscond	cond_eq_wy	infres	resample_abscond	cond_abscond	cond_eq_best	asy_bh		
under-1 fit item	mis-150 fit at 0	63		54	53	55	50		41	49	42	7
under-1 fit item	mis-400 fit at 0	86		77	79	78	83		72	72	73	62
under-1 fit item	mis-150 fit at -2	33		27	24	23	29		23	20	18	1
under-1 fit item	mis-400 fit at -2	77		71	74	68	69		59	62	55	35
under-2 fit items	mis-150 fit at 0	1		1	1	0	1		1	1	0	0
under-2 fit items	mis-400 fit at 0	8		5	5	10	4		1	2	6	0
under-2 fit items	mis-150 fit at -2	0		0	0	0	1		0	0	0	0
under-2 fit items	mis-400 fit at -2	3		6	5	3	3		6	4	3	0

Pooled detection

```
long |>
  filter(misfit) |>
  group_by(direction, method) |>
  summarise(detected = round(100 * mean(flag == direction), 1),
            .groups = "drop") |>
  pivot_wider(names_from = method, values_from = detected)
```

direction	infres	resample_abscond	cond_abscond	cond_eq_wy	infres	resample_abscond	cond_abscond	cond_eq_best	asy_bh		
over-fit	37.7		47.0	45.0	48.4	37.8		45.4	44.3	43.8	47.9
under-fit	37.3		37.7	36.9	36.9	35.0		35.3	35.5	31.8	14.4

Iterations kept

Simulated datasets with an unobserved category are rejected, so a cutoff can rest on fewer than the requested 400 iterations.

```
new |>
  distinct(cell, rep, iterations) |>
  left_join(run_cells |> select(cell, n, target), by = "cell") |>
  group_by(n, target) |>
  summarise(datasets = n(),
            below_400 = round(100 * mean(iterations < 400), 1),
            below_300 = round(100 * mean(iterations < 300), 1),
            min = min(iterations), .groups = "drop")
```

n	target	datasets	below_400	below_300	min
150	-2	400	79.2	20.5	156
150	0	400	33.8	2.5	247
400	-2	400	20.8	0.8	246
400	0	400	0.5	0.0	398

Rescued datasets

Detection with the shipped configuration in the datasets that failed in the earlier study.

```
new |>
  semi_join(rescued, by = c("cell", "rep")) |>
  left_join(run_cells |> select(cell, direction), by = "cell") |>
  filter(misfit) |>
  group_by(cell) |>
  summarise(misfit_items = n(),
            detected = round(100 * mean(cond_eq_wy == direction)),
            .groups = "drop")
```

cell	misfit_items	detected
overfit_1item_t0_n150	1	0
overfit_1item_tm2_n150	10	30
overfit_1item_tm2_n400	1	100
overfit_2item_t0_n150	2	0
overfit_2item_tm2_n150	28	18
underfit_1item_tm2_n150	5	40
underfit_2item_tm2_n150	8	0

Interpretation

The equal-tailed arm is withdrawn. The `cond_eq_*` columns come from the first equal-tailed version, whose Westfall-Young step-down turned out to be liberal (`restscore_eti_null.qmd`), and the approach was then dropped from the package. They are kept in the tables as a record and not interpreted. `RMitemRestscore()` ships with the conditional DGP and the two-sided $|t|$ p-value, which is `cond_abs_wy` here.

The results rest on 100 datasets per cell. Differences of a few points within one cell are noise, and the statements below rely on the pooled paired comparisons.

1. Every dataset ran. All 1,600 datasets gave a `restscore` result, with no errors. The 36 datasets in these cells that failed in the earlier study, where a category was used only by respondents with a zero or

perfect total score, all ran with the conditional DGP, because it reproduces those respondents' patterns. They come from the hardest cells, and detection in them was low (0 to 40 percent in the four cells with five or more misfitting items).

2. The DGP changes little, and mainly off target. With the $|t|$ p-value, the conditional DGP detected overfitting items at -2 logits 4.5 points less often than the resample DGP (6 against 31 discordant items). In the other three direction and targeting combinations the two were within 1.4 points. This is the direction expected from the null study, where resample was about 1.4 points liberal.

3. Against infit, the shipped test is more sensitive to overfit and about equal for underfit. Conditional WY detected overfitting items 7.8 points more often than infit WY off target (71 against 27 discordant items) and 6.9 points more often on target (49 against 8). For underfit the two were close (+1.4 off target, -2.2 on target). Pooled, overfit detection was 45.0 against 37.7 percent and underfit 36.9 against 37.3. The conclusions of `restscore_infit_power.qmd` hold with the restscore test at its nominal level.

4. Against the asymptotic iarm test, the shipped test recovers underfit. Conditional WY detected underfitting items 22.3 points more often than asymptotic BH off target and 22.7 points more often on target, and iarm detected no underfitting item that the bootstrap missed (131 and 136 against 0 discordant items). For overfit, iarm was ahead off target by 6.6 points (9 against 46) and level on target. Part of iarm's overfit lead comes from its liberal overfit tail at small n , which also produces false overfit flags under the null.

5. The cascade. With an underfitting item planted, the conditional test flagged 1.9 (one item) and 2.4 percent (two items) of the fitting items as overfit, against 1.1 and 1.2 for infit. With overfitting items planted, false underfit flags were 1.4 and 5.9 percent, against 1.5 and 6.5 for infit. In clean decisions restscore is mostly ahead of infit when the planted misfit is overfit and mostly behind when it is underfit, for example 79 against 86 percent with one underfitting item on target at $n = 400$. The asymptotic test has the most clean decisions in the overfit cells, because its underfit tail almost never rejects, and the fewest in the underfit cells (7 against 53 percent with one item on target at $n = 150$).

6. The conditional DGP loses iterations in small, off-target samples. At $n = 150$ with the misfitting items at -2 logits, 79 percent of datasets kept fewer than 400 iterations, 20.5 percent fewer than 300, and one only 156. The simulated datasets that are rejected are those missing a category, which the conditional DGP produces more often when extreme categories are sparse. For infit, fewer than 400 iterations made WY mildly liberal. For restscore this has not been measured.

Limitations. Infit still uses the resample DGP, which the FWER paper validated for infit.

Follow-up: split one-sided step-downs

Added 2026-10-02. `restscore_eti_null.qmd` tested an equal-tailed family-wise test built from two one-sided Westfall-Young step-downs, one per direction, each at $\alpha/2$. With the mean, SD or quantiles computed from all $B + 1$ values (pooled), it held the family-wise rate and split it evenly between the directions, on synthetic nulls and on the real restscore null. This section measures its power on the same 1,600 datasets.

Variants, all from one cutoff simulation per dataset:

- `abs_t`: the shipped $|t|$. It must reproduce `cond_abs_wy` above.
- `abs_t_pooled`: $|t|$ with pooled moments.
- `split_t`, `split_t_pooled`: t in the overfit run, $-t$ in the underfit run.
- `split_two_piece`, `split_two_piece_pooled`: each run scaled by its own tail of the null, (97.5th percentile $-$ median) for overfit and (median $-$ 2.5th percentile) for underfit.

The pooled split variants are the candidates. The simulated-only versions are included to show what pooling costs in power. The observed differences and simulated matrices of every dataset are saved, so later variants can be checked without rerunning the bootstrap.

Cost. About 1.5 hours on 10 cores, as for the run above. The run is off by default. Set `RUN_SPLIT_POWER <- TRUE` to start it. Results are saved per cell, and the result chunks run once all 16 cells are saved.

```

RUN_SPLIT_POWER <- TRUE
SPLIT_POWER_DIR <- "restscore_power_split"
split_power_files <- file.path(SPLIT_POWER_DIR,
                                paste0(run_cells$cell, ".rds"))
SPLIT_POWER_READY <- all(file.exists(split_power_files))
split_variant_levels <- c("abs_t", "abs_t_pooled", "split_t",
                           "split_t_pooled", "split_two_piece",
                           "split_two_piece_pooled")

# Copy of split_null_variants() in restscore_eti_null.qmd: adjusted p-values
# of every variant for one dataset, with the arithmetic of
# .bootstrap_pvalues().
split_variants <- function(obs, sim) {
  sim <- sim[, names(obs), drop = FALSE]
  B <- nrow(sim)
  guard <- function(x) {
    x[!is.finite(x) | x < 1e-8] <- NA_real_
    x
  }
  out <- data.frame(item = names(obs), stringsAsFactors = FALSE)
  for (pooled in c(FALSE, TRUE)) {
    sfx <- if (pooled) "_pooled" else ""
    ref <- if (pooled) rbind(obs, sim) else sim
    m <- colMeans(ref, na.rm = TRUE)
    s <- guard(apply(ref, 2L, stats::sd, na.rm = TRUE))
    t_obs <- (obs - m) / s
    t_sim <- sweep(sweep(sim, 2L, m, `-`), 2L, s, `/`)
    q <- apply(ref, 2L, stats::quantile, probs = c(.025, .5, .975),
              na.rm = TRUE, names = FALSE)
    up <- guard(q[3, ] - q[2, ])
    dn <- guard(q[2, ] - q[1, ])
    dev_sim <- sweep(sim, 2L, q[2, ], `-`)
    out[[paste0("t", sfx)]] <- t_obs
    out[[paste0("padj_abs", sfx)]] <- .wy_stepdown(abs(t_obs), abs(t_sim), B)
    out[[paste0("padj_t_hi", sfx)]] <- .wy_stepdown(t_obs, t_sim, B)
    out[[paste0("padj_t_lo", sfx)]] <- .wy_stepdown(-t_obs, -t_sim, B)
    out[[paste0("padj_tp_hi", sfx)]] <- .wy_stepdown(
      (obs - q[2, ]) / up, sweep(dev_sim, 2L, up, `/`), B
    )
    out[[paste0("padj_tp_lo", sfx)]] <- .wy_stepdown(
      (q[2, ] - obs) / dn, sweep(-dev_sim, 2L, dn, `/`), B
    )
  }
  out
}

# Flags ("overfit", "underfit" or "") per variant, from split_variants().
split_flags <- function(v, alpha = 0.05) {
  two <- function(padj, t) {
    ifelse(!is.na(padj) & padj < alpha,
           ifelse(t > 0, "overfit", "underfit"), "")
  }

```

```

}
one <- function(lo, hi) {
  ifelse(!is.na(hi) & hi < alpha / 2, "overfit",
        ifelse(!is.na(lo) & lo < alpha / 2, "underfit", ""))
}
data.frame(
  abs_t = two(v$padj_abs, v$t),
  abs_t_pooled = two(v$padj_abs_pooled, v$t_pooled),
  split_t = one(v$padj_t_lo, v$padj_t_hi),
  split_t_pooled = one(v$padj_t_lo_pooled, v$padj_t_hi_pooled),
  split_two_piece = one(v$padj_tp_lo, v$padj_tp_hi),
  split_two_piece_pooled = one(v$padj_tp_lo_pooled, v$padj_tp_hi_pooled),
  stringsAsFactors = FALSE
)
}

# Same data, seed and cutoff call as one_dataset().
one_dataset_split <- function(rep, cell_row) {
  setup <- item_setup(cell_row$n_misfit, cell_row$target)
  seed <- 1e6 * cell_row$row + rep
  d <- sim_power_data(
    n = cell_row$n, n_misfit = cell_row$n_misfit, target = cell_row$target,
    direction = cell_row$direction, rho = RHO, slope = SLOPE, seed = seed
  )

  tryCatch({
    cr <- suppressWarnings(suppressMessages(RMitemRestscoreCutoff(
      d, iterations = B, parallel = FALSE, seed = seed,
      cutoff_method = "quantile", dgp = "conditional"
    )))
    fw <- suppressWarnings(suppressMessages(RMitemRestscore(
      d, cutoff = cr, output = "dataframe"
    )))
    sim_mat <- tapply(cr$results$Difference,
                      list(cr$results$iteration, cr$results$Item),
                      function(x) x[1L])
    obs <- stats::setNames(fw$Difference, fw$Item)
    v <- split_variants(obs, sim_mat)
    fl <- split_flags(v)[match(names(d), v$item), ]
    list(
      table = data.frame(
        rep = rep, item = names(d),
        misfit = seq_len(7) %in% setup$misfit,
        pkg_wy = fw$Flagged[match(names(d), fw$Item)],
        fl, iterations = cr$actual_iterations,
        row.names = NULL, stringsAsFactors = FALSE
      ),
      sims = list(rep = rep, obs = obs,
                  sim = sim_mat[, names(obs), drop = FALSE])
    )
  }, error = function(e) {
    list(error = data.frame(rep = rep, error = conditionMessage(e)))
  })
}

```

```

dir.create(SPLIT_POWER_DIR, showWarnings = FALSE)
for (i in seq_len(nrow(run_cells))) {
  if (file.exists(split_power_files[i])) next
  t0 <- Sys.time()
  res <- mclapply(seq_len(R_DATASETS),
                  function(r) one_dataset_split(r, run_cells[i, ]),
                  mc.cores = NCORES, mc.preschedule = FALSE)
  # A worker that dies returns a try-error, kept as an error row.
  res <- lapply(res, function(x) {
    if (is.list(x)) x else list(error = data.frame(rep = NA, error = as.character(x)))
  })
  ok <- vapply(res, function(x) is.null(x$error), logical(1))
  # Simulated values first, so a saved result implies saved simulations.
  saveRDS(lapply(res[ok], `[[`, "sims"),
           file.path(SPLIT_POWER_DIR, paste0(run_cells$cell[i], "_sims.rds")))
  saveRDS(list(res = lapply(res, function(x) {
    if (is.null(x$error)) x$table else x$error
  })),
           elapsed_min = as.numeric(Sys.time() - t0, units = "mins"),
           split_power_files[i])
}
SPLIT_POWER_READY <- all(file.exists(split_power_files))

```

```

sp_raw <- lapply(run_cells$cell, function(cl) {
  x <- readRDS(file.path(SPLIT_POWER_DIR, paste0(cl, ".rds")))
  is_ok <- vapply(x$res, function(r) "item" %in% names(r), logical(1))
  list(ok = cbind(cell = cl, do.call(rbind, x$res[is_ok])),
       err = if (any(!is_ok)) cbind(cell = cl, do.call(rbind, x$res[!is_ok])),
       minutes = round(x$elapsed_min, 1))
})
sp <- do.call(rbind, lapply(sp_raw, `[[`, "ok")) |>
  left_join(run_cells |> select(cell, n, target, n_misfit, direction),
            by = "cell")
sp_errors <- do.call(rbind, lapply(sp_raw, `[[`, "err"))
data.frame(
  cell = run_cells$cell,
  datasets = vapply(sp_raw, function(x) length(unique(x$ok$rep)), integer(1)),
  errors = vapply(sp_raw, function(x) {
    if (is.null(x$err)) 0L else nrow(x$err)
  }, integer(1)),
  minutes = vapply(sp_raw, `[[`, numeric(1), "minutes")
)

```

cell	datasets	errors	minutes
underfit_1item_t0_n150	100	0	5.0
underfit_1item_t0_n400	100	0	6.0
underfit_1item_tm2_n150	100	0	4.7
underfit_1item_tm2_n400	100	0	6.0
underfit_2item_t0_n150	100	0	4.9
underfit_2item_t0_n400	100	0	6.0

cell	datasets	errors	minutes
underfit_2item_tm2_n150	100	0	4.5
underfit_2item_tm2_n400	100	0	5.9
overfit_1item_t0_n150	100	0	4.9
overfit_1item_t0_n400	100	0	6.0
overfit_1item_tm2_n150	100	0	4.6
overfit_1item_tm2_n400	100	0	6.0
overfit_2item_t0_n150	100	0	4.9
overfit_2item_t0_n400	100	0	6.0
overfit_2item_tm2_n150	100	0	4.5
overfit_2item_tm2_n400	100	0	6.0

```
if (!is.null(sp_errors)) count(sp_errors, error)
```

Checks

`abs_t` against the package flags of this run and against the cached `cond_abs_wy`. Both mismatch counts should be 0.

```
sp |>
  inner_join(new |> select(cell, rep, item, cond_abs_wy),
             by = c("cell", "rep", "item")) |>
  summarise(items = n(),
            mismatch_package = sum(abs_t != pkg_wy),
            mismatch_cached = sum(abs_t != cond_abs_wy))
```

items	mismatch_package	mismatch_cached
11200	0	0

Detection of the misfitting items

Proportion of misfitting items flagged in the correct direction, per cell (100 datasets) and pooled over sample size and the number of misfitting items.

```
sp_long <- sp |>
  pivot_longer(all_of(split_variant_levels), names_to = "variant",
               values_to = "flag") |>
  mutate(variant = factor(variant, levels = split_variant_levels),
         targeting = ifelse(target == 0, "at 0", "at -2"))
sp_long |>
  filter(misfit) |>
  group_by(direction, n_misfit, targeting, n, variant) |>
  summarise(correct = round(100 * mean(flag == direction)),
            .groups = "drop") |>
  pivot_wider(names_from = variant, values_from = correct) |>
  arrange(direction, n_misfit, desc(targeting), n)
```

direction	n_misfit	targeting	nabs	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled	
overfit	1	at 0	150	29	29	30	29	32	27
overfit	1	at 0	400	84	84	86	86	87	85
overfit	1	at -2	150	20	20	22	22	27	20
overfit	1	at -2	400	76	75	75	75	76	76
overfit	2	at 0	150	24	24	28	26	28	26
overfit	2	at 0	400	68	68	70	68	67	66
overfit	2	at -2	150	12	11	12	11	14	12
overfit	2	at -2	400	56	56	56	55	56	54
underfit	1	at 0	150	61	60	59	59	57	56
underfit	1	at 0	400	97	96	96	96	96	96
underfit	1	at -2	150	28	28	27	27	27	25
underfit	1	at -2	400	83	83	78	78	77	76
underfit	2	at 0	150	12	12	11	11	11	10
underfit	2	at 0	400	34	34	32	31	32	32
underfit	2	at -2	150	10	9	7	7	7	6
underfit	2	at -2	400	29	28	26	26	28	26

```

sp_long |>
  filter(misfit) |>
  group_by(direction, targeting, variant) |>
  summarise(correct = round(100 * mean(flag == direction), 1),
            .groups = "drop") |>
  pivot_wider(names_from = variant, values_from = correct)

```

direction	target- ing	abs_tabs_t_pooled	split_t	split_t_pooled	split_two_pieces	split_two_piece_pooled	
overfit	at -2	38.3	38.0	38.7	38.2	40.3	38.2
overfit	at 0	49.7	49.5	51.7	50.8	51.3	49.3
under- fit	at -2	31.5	31.0	28.7	28.7	29.0	27.8
under- fit	at 0	42.0	41.2	40.0	39.8	39.8	39.2

Paired comparisons

Misfitting items detected by one variant and not the other, pooled over sample size and the number of misfitting items. `difference_pts` is the first variant minus the second.

```
sp_paired <- function(a, b) {
  sp |>
    filter(misfit) |>
    mutate(hit_a = .data[[a]] == direction, hit_b = .data[[b]] == direction) |>
    group_by(direction, targeting = ifelse(target == 0, "at 0", "at -2")) |>
    summarise(items = n(), a_only = sum(hit_a & !hit_b),
              b_only = sum(!hit_a & hit_b),
              difference_pts = round(100 * (mean(hit_a) - mean(hit_b)), 1),
              .groups = "drop") |>
    mutate(comparison = paste(a, "vs", b), .before = 1)
}
bind_rows(
  sp_paired("split_t_pooled", "abs_t"),
  sp_paired("split_two_piece_pooled", "abs_t"),
  sp_paired("split_two_piece_pooled", "split_t_pooled"),
  sp_paired("split_t_pooled", "split_t"),
  sp_paired("abs_t_pooled", "abs_t")
)
```

comparison	direction	targeting	items	a_only	b_only	difference_pts
split_t_pooled vs abs_t	overfit	at -2	600	6	7	-0.2
split_t_pooled vs abs_t	overfit	at 0	600	12	5	1.2
split_t_pooled vs abs_t	underfit	at -2	600	0	17	-2.8
split_t_pooled vs abs_t	underfit	at 0	600	0	13	-2.2
split_two_piece_pooled vs abs_t	overfit	at -2	600	16	17	-0.2
split_two_piece_pooled vs abs_t	overfit	at 0	600	13	15	-0.3
split_two_piece_pooled vs abs_t	underfit	at -2	600	0	22	-3.7
split_two_piece_pooled vs abs_t	underfit	at 0	600	2	19	-2.8

comparison	direction	targeting	items	a_only	b_only	difference_pts
split_two_piece_pooled vs split_t_pooled	overfit	at -2	600	15	15	0.0
split_two_piece_pooled vs split_t_pooled	overfit	at 0	600	7	16	-1.5
split_two_piece_pooled vs split_t_pooled	underfit	at -2	600	6	11	-0.8
split_two_piece_pooled vs split_t_pooled	underfit	at 0	600	3	7	-0.7
split_t_pooled vs split_t	overfit	at -2	600	0	3	-0.5
split_t_pooled vs split_t	overfit	at 0	600	0	5	-0.8
split_t_pooled vs split_t	underfit	at -2	600	0	0	0.0
split_t_pooled vs split_t	underfit	at 0	600	0	1	-0.2
abs_t_pooled vs abs_t	overfit	at -2	600	0	2	-0.3
abs_t_pooled vs abs_t	overfit	at 0	600	0	1	-0.2
abs_t_pooled vs abs_t	underfit	at -2	600	0	3	-0.5
abs_t_pooled vs abs_t	underfit	at 0	600	0	5	-0.8

False flags on the fitting items, by direction

```
sp_long |>
  filter(!misfit) |>
  group_by(direction, n_misfit, variant) |>
  summarise(false_overfit = round(100 * mean(flag == "overfit"), 1),
            false_underfit = round(100 * mean(flag == "underfit"), 1),
            .groups = "drop") |>
  pivot_wider(names_from = n_misfit,
              values_from = c(false_overfit, false_underfit),
              names_prefix = "k")
```

direction	variant	false_overfit_k1	false_overfit_k2	false_underfit_k1	false_underfit_k2
overfit	abs_t	0.2	0.0	1.4	5.7
overfit	abs_t_pooled	0.1	0.0	1.2	5.4
overfit	split_t	0.2	0.0	0.9	4.0
overfit	split_t_pooled	0.2	0.0	0.9	3.8
overfit	split_two_piece	0.2	0.0	0.8	4.5
overfit	split_two_piece_pooled	0.1	0.0	0.7	4.0
underfit	abs_t	1.8	2.4	0.0	0.1

direction	variant	false_overfit_k1	false_overfit_k2	false_underfit_k1	false_underfit_k2
under-fit	abs_t_pooled	1.7	2.1	0.0	0.1
under-fit	split_t	1.7	2.2	0.0	0.1
under-fit	split_t_pooled	1.6	2.1	0.0	0.1
under-fit	split_two_piece	2.1	2.4	0.0	0.1
under-fit	split_two_piece_pooled	1.8	2.0	0.0	0.1

Clean decisions

Every misfitting item flagged in the correct direction and no fitting item flagged, pooled over sample size.

```
sp_long |>
  group_by(direction, n_misfit, targeting, n, variant, rep) |>
  summarise(clean = all(flag[misfit] == direction[1]) &
    all(flag[!misfit] == ""), .groups = "drop") |>
  group_by(direction, n_misfit, targeting, variant) |>
  summarise(clean = round(100 * mean(clean)), .groups = "drop") |>
  pivot_wider(names_from = variant, values_from = clean) |>
  arrange(direction, n_misfit, desc(targeting))
```

direction	n_misfit	targeting	abs	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
over-fit	1	at 0	49	50	54	54	56	53
over-fit	1	at -2	44	44	46	46	48	46
over-fit	2	at 0	12	12	16	17	14	14
over-fit	2	at -2	8	9	13	12	12	12
underfit	1	at 0	66	66	67	66	63	65
underfit	1	at -2	50	50	47	48	46	46
underfit	2	at 0	3	4	3	4	4	4
underfit	2	at -2	2	2	2	2	2	2

Extension to 300 datasets per cell

The cells above rest on 100 datasets, so a single cell's detection rate has a Monte Carlo SE of up to 5 points. This extension adds datasets 101 to 300 in the same 16 cells, with the same seed scheme ($1e6 * \text{row} + \text{rep}$), so the combined 300 datasets per cell are a continuation of the first 100. Infit and the asymptotic test are not extended, so comparisons with them stay at 100 datasets.

Cost. About 3 hours on 10 cores. Off by default. Set `RUN_SPLIT_POWER_EXT <- TRUE` to start it.

```
RUN_SPLIT_POWER_EXT <- TRUE
REPS_EXT <- 101:300
SPLIT_EXT_DIR <- "restscore_power_split_ext"
split_ext_files <- file.path(SPLIT_EXT_DIR, paste0(run_cells$cell, ".rds"))
SPLIT_EXT_READY <- SPLIT_POWER_READY && all(file.exists(split_ext_files))

dir.create(SPLIT_EXT_DIR, showWarnings = FALSE)
for (i in seq_len(nrow(run_cells))) {
  if (file.exists(split_ext_files[i])) next
  t0 <- Sys.time()
  res <- mclapply(REPS_EXT,
                 function(r) one_dataset_split(r, run_cells[i, ]),
                 mc.cores = NCORES, mc.preschedule = FALSE)
  res <- lapply(res, function(x) {
    if (is.list(x)) x else list(error = data.frame(rep = NA, error = as.character(x)))
  })
  ok <- vapply(res, function(x) is.null(x$error), logical(1))
  saveRDS(lapply(res[ok], `[[`, "sims"),
          file.path(SPLIT_EXT_DIR, paste0(run_cells$cell[i], "_sims.rds")))
  saveRDS(list(res = lapply(res, function(x) {
    if (is.null(x$error)) x$table else x$error
  })),
          elapsed_min = as.numeric(Sys.time() - t0, units = "mins"),
          split_ext_files[i])
}
SPLIT_EXT_READY <- SPLIT_POWER_READY && all(file.exists(split_ext_files))

ext_raw <- lapply(run_cells$cell, function(cl) {
  x <- readRDS(file.path(SPLIT_EXT_DIR, paste0(cl, ".rds")))
  is_ok <- vapply(x$res, function(r) "item" %in% names(r), logical(1))
  list(ok = cbind(cell = cl, do.call(rbind, x$res[is_ok])),
       err = if (any(!is_ok)) cbind(cell = cl, do.call(rbind, x$res[!is_ok])),
       minutes = round(x$elapsed_min, 1))
})
sp300 <- bind_rows(
  sp |> select(-n, -target, -n_misfit, -direction),
  do.call(rbind, lapply(ext_raw, `[[`, "ok"))
) |>
  left_join(run_cells |> select(cell, n, target, n_misfit, direction),
            by = "cell")
ext_errors <- do.call(rbind, lapply(ext_raw, `[[`, "err"))
data.frame(
  cell = run_cells$cell,
  datasets_ext = vapply(ext_raw, function(x) length(unique(x$ok$rep)),
                        integer(1)),
```

```

errors = vapply(ext_raw, function(x) {
  if (is.null(x$err)) 0L else nrow(x$err)
}, integer(1)),
minutes = vapply(ext_raw, `[`, numeric(1), "minutes"),
datasets_total = vapply(run_cells$cell, function(cl) {
  length(unique(sp300$rep[sp300$cell == cl]))
}, integer(1))
)

```

	cell	datasets_ext	er- rors	min- utes	datasets_total
underfit_1item_t0_n150	underfit_1item_t0_n150	200	0	9.7	300
underfit_1item_t0_n400	underfit_1item_t0_n400	200	0	12.1	300
underfit_1item_tm2_n150	underfit_1item_tm2_n150	200	0	9.3	300
underfit_1item_tm2_n400	underfit_1item_tm2_n400	200	0	12.0	300
underfit_2item_t0_n150	underfit_2item_t0_n150	200	0	9.7	300
underfit_2item_t0_n400	underfit_2item_t0_n400	200	0	12.0	300
underfit_2item_tm2_n150	underfit_2item_tm2_n150	200	0	8.9	300
underfit_2item_tm2_n400	underfit_2item_tm2_n400	200	0	11.9	300
overfit_1item_t0_n150	overfit_1item_t0_n150	200	0	9.7	300
overfit_1item_t0_n400	overfit_1item_t0_n400	200	0	11.9	300
overfit_1item_tm2_n150	overfit_1item_tm2_n150	200	0	9.2	300
overfit_1item_tm2_n400	overfit_1item_tm2_n400	200	0	12.0	300
overfit_2item_t0_n150	overfit_2item_t0_n150	200	0	9.8	300
overfit_2item_t0_n400	overfit_2item_t0_n400	200	0	11.9	300
overfit_2item_tm2_n150	overfit_2item_tm2_n150	200	0	8.9	300
overfit_2item_tm2_n400	overfit_2item_tm2_n400	200	0	12.0	300

```

if (!is.null(ext_errors)) count(ext_errors, error)
# The package flags must still agree with the local |t|.
sum(sp300$abs_t != sp300$pkg_wy)

```

```
[1] 0
```

Detection, 300 datasets per cell

```

sp300_long <- sp300 |>
  pivot_longer(all_of(split_variant_levels), names_to = "variant",
               values_to = "flag") |>
  mutate(variant = factor(variant, levels = split_variant_levels),
         targeting = ifelse(target == 0, "at 0", "at -2"))
sp300_long |>
  filter(misfit) |>
  group_by(direction, n_misfit, targeting, n, variant) |>
  summarise(correct = round(100 * mean(flag == direction)),

```

```

    .groups = "drop") |>
  pivot_wider(names_from = variant, values_from = correct) |>
  arrange(direction, n_misfit, desc(targeting), n)

```

direction	n_misfit	targeting	nabs	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
over-fit	1	at 0	150	33	33	34	34	35
over-fit	1	at 0	400	87	87	88	87	89
over-fit	1	at -2	150	20	19	22	21	26
over-fit	1	at -2	400	76	75	78	78	77
over-fit	2	at 0	150	22	22	24	24	24
over-fit	2	at 0	400	75	74	76	75	75
over-fit	2	at -2	150	14	14	16	15	18
over-fit	2	at -2	400	57	56	56	55	57
underfit	1	at 0	150	59	58	57	56	55
underfit	1	at 0	400	97	96	96	96	97
underfit	1	at -2	150	33	32	30	29	29
underfit	1	at -2	400	74	73	71	70	71
underfit	2	at 0	150	11	11	10	10	10
underfit	2	at 0	400	36	35	33	32	34
underfit	2	at -2	150	9	9	7	7	7
underfit	2	at -2	400	29	28	27	26	26

```

sp300_long |>
  filter(misfit) |>
  group_by(direction, targeting, variant) |>
  summarise(correct = round(100 * mean(flag == direction), 1),

```

```
.groups = "drop") |>
pivot_wider(names_from = variant, values_from = correct)
```

direction	target- ing	abs_tabs_t_pooled	split_t	split_t_pooled	split_two_pieces	split_two_piece_pooled	
overfit	at -2	39.6	38.9	40.5	39.8	42.2	40.3
overfit	at 0	52.7	51.8	53.9	53.2	53.3	51.6
under- fit	at -2	30.6	29.9	28.2	27.6	27.8	26.8
under- fit	at 0	41.5	40.8	39.7	39.3	39.9	38.6

Paired comparisons, 300 datasets per cell

```
sp300_paired <- function(a, b) {
  sp300 |>
    filter(misfit) |>
    mutate(hit_a = .data[[a]] == direction, hit_b = .data[[b]] == direction) |>
    group_by(direction, targeting = ifelse(target == 0, "at 0", "at -2")) |>
    summarise(items = n(), a_only = sum(hit_a & !hit_b),
              b_only = sum(!hit_a & hit_b),
              difference_pts = round(100 * (mean(hit_a) - mean(hit_b)), 1),
              se_pts = round(100 * sqrt(a_only + b_only) / n(), 1),
              .groups = "drop") |>
    mutate(comparison = paste(a, "vs", b), .before = 1)
}
bind_rows(
  sp300_paired("split_t_pooled", "abs_t"),
  sp300_paired("split_two_piece_pooled", "abs_t"),
  sp300_paired("split_two_piece_pooled", "split_t_pooled"),
  sp300_paired("split_t_pooled", "split_t"),
  sp300_paired("abs_t_pooled", "abs_t")
)
```

comparison	direction	targeting	items	a_only	b_only	difference_pts	se_pts
split_t_pooled vs abs_t	overfit	at -2	1800	25	21	0.2	0.4
split_t_pooled vs abs_t	overfit	at 0	1800	30	21	0.5	0.4
split_t_pooled vs abs_t	underfit	at -2	1800	1	54	-2.9	0.4
split_t_pooled vs abs_t	underfit	at 0	1800	0	40	-2.2	0.4
split_two_piece_pooled vs abs_t	overfit	at -2	1800	54	41	0.7	0.5
split_two_piece_pooled vs abs_t	overfit	at 0	1800	30	50	-1.1	0.5
split_two_piece_pooled vs abs_t	underfit	at -2	1800	6	74	-3.8	0.5
split_two_piece_pooled vs abs_t	underfit	at 0	1800	9	61	-2.9	0.5

comparison		direction	target-ing	itemsa_only	b_only	difference_pts	se_pts	
split_two_piece_pooled split_t_pooled	vs	overfit	at -2	1800	46	37	0.5	0.5
split_two_piece_pooled split_t_pooled	vs	overfit	at 0	1800	22	51	-1.6	0.5
split_two_piece_pooled split_t_pooled	vs	under-fit	at -2	1800	19	34	-0.8	0.4
split_two_piece_pooled split_t_pooled	vs	under-fit	at 0	1800	17	29	-0.7	0.4
split_t_pooled vs split_t		overfit	at -2	1800	0	12	-0.7	0.2
split_t_pooled vs split_t		overfit	at 0	1800	0	13	-0.7	0.2
split_t_pooled vs split_t		under-fit	at -2	1800	0	10	-0.6	0.2
split_t_pooled vs split_t		under-fit	at 0	1800	0	7	-0.4	0.1
abs_t_pooled vs abs_t		overfit	at -2	1800	0	12	-0.7	0.2
abs_t_pooled vs abs_t		overfit	at 0	1800	0	15	-0.8	0.2
abs_t_pooled vs abs_t		under-fit	at -2	1800	0	12	-0.7	0.2
abs_t_pooled vs abs_t		under-fit	at 0	1800	0	12	-0.7	0.2

False flags and clean decisions, 300 datasets per cell

```
sp300_long |>
  filter(!misfit) |>
  group_by(direction, n_misfit, variant) |>
  summarise(false_overfit = round(100 * mean(flag == "overfit"), 1),
            false_underfit = round(100 * mean(flag == "underfit"), 1),
            .groups = "drop") |>
  pivot_wider(names_from = n_misfit,
              values_from = c(false_overfit, false_underfit),
              names_prefix = "k")
```

direction	variant	false_overfit_k1	false_overfit_k2	false_underfit_k1	false_underfit_k2
overfit	abs_t	0.1	0.0	1.6	6.2
overfit	abs_t_pooled	0.1	0.0	1.4	6.0
overfit	split_t	0.2	0.0	1.1	4.8
overfit	split_t_pooled	0.1	0.0	1.0	4.7
overfit	split_two_piece	0.2	0.0	1.1	5.1
overfit	split_two_piece_pooled	0.1	0.0	1.0	4.6
under-fit	abs_t	2.1	2.3	0.0	0.0

direction	variant	false_overfit_k1	false_overfit_k2	false_underfit_k1	false_underfit_k2
under-fit	abs_t_pooled	1.9	2.2	0.0	0.0
under-fit	split_t	2.0	2.3	0.0	0.0
under-fit	split_t_pooled	1.9	2.2	0.0	0.0
under-fit	split_two_piece	2.1	2.6	0.0	0.0
under-fit	split_two_piece_pooled	1.8	2.2	0.0	0.0

```
sp300_long |>
  group_by(direction, n_misfit, targeting, n, variant, rep) |>
  summarise(clean = all(flag[misfit] == direction[1]) &
    all(flag[!misfit] == ""), .groups = "drop") |>
  group_by(direction, n_misfit, targeting, variant) |>
  summarise(clean = round(100 * mean(clean)), .groups = "drop") |>
  pivot_wider(names_from = variant, values_from = clean) |>
  arrange(direction, n_misfit, desc(targeting))
```

direction	n_misfit	targeting	abs_t	abs_t_pooled	split_t	split_t_pooled	split_two_piece	split_two_piece_pooled
over-fit	1	at 0	51	52	55	55	56	54
over-fit	1	at -2	42	42	46	46	46	45
over-fit	2	at 0	14	13	18	18	16	16
over-fit	2	at -2	9	10	13	12	12	12
underfit	1	at 0	64	64	64	64	63	63
underfit	1	at -2	48	47	45	45	44	43
underfit	2	at 0	3	4	3	3	3	3
underfit	2	at -2	2	2	2	2	2	2

Interpretation of the split follow-up

All 4,800 datasets ran without errors, 100 per cell in the first run and 200 more in the extension. The local `|t|` reproduced the package flags and the cached `cond_abs_wy` on every item, so the comparisons below differ only in the p-value. The figures are from the 300 datasets per cell, 1,800 misfitting items per direction and targeting.

1. The split costs underfit detection. `split_t_pooled` detected underfitting items 2.9 points less often than the shipped `|t|` off target and 2.2 points less often on target (SE 0.4), and almost never caught an item that `|t|` missed (1 against 54 and 0 against 40 discordant items). The first 100 datasets gave 2.8 and 2.2.

2. It gains no overfit detection. The differences were +0.2 off target and +0.5 on target (SE 0.4, 25 against 21 and 30 against 21 discordant items). The +1.2 on target in the first 100 datasets did not hold up. The α moved to the overfit tail buys little extra overfit power in these cells.

3. It makes fewer false underfit flags. With overfitting items planted, fitting items were flagged as underfit 1.0 (one item) and 4.7 percent (two items) of the time, against 1.6 and 6.2 for `|t|`. False overfit flags with underfit planted were about equal (1.9 and 2.2 against 2.1 and 2.3).

4. Clean decisions improve in the overfit cells. Every misfitting item flagged in the correct direction and no fitting item flagged: 55 against 51 and 46 against 42 percent with one overfitting item on and off target, 18 against 14 and 12 against 9 with two. In the underfit cells the split was level or 1 to 3 points behind (45 against 48 with one item off target).

5. Two-piece scaling does not help. `split_two_piece_pooled` was behind `split_t_pooled` in three of four comparisons, by up to 1.6 points (overfit on target, 22 against 51 discordant items). Together with the narrow spread of null skewness across items in the real null, this leaves no case for it.

6. Pooling costs little. Pooled moments lowered detection by 0.4 to 0.8 points for both `|t|` and the split, always in the same direction (0 against 7 to 15 discordant items), in return for an exact level under exchangeability.

The trade. Against the shipped `|t|`, `split_t_pooled` gives equal tails under the null (2.40 against 2.75 percent of datasets with a flag in each direction on the real null, where `|t|` gives 3.70 and 1.55) and fewer false underfit flags when items overfit. It costs 2 to 3 points of underfit detection and gains no overfit detection. Whether to ship it is a decision about which error matters more, not a question the simulations settle.